

TFD-Agent: Autonomous Software Engineering via Test-Feedback in Gemma 4

Raja Rajak

Lead Researcher & Systems Architect • Independent AI Researcher • Kaggle: @rajrajak99

Open-Source Codebase: github.com/rajrajak99/gemma4-tfd-agent

Submitted to: Google — The Gemma 4 Developer Agent Paper Track (Kaggle 2026)

Abstract

Autonomous software engineering agents increasingly rely on massive, closed-source foundation models hosted behind cloud APIs. While capable, these models incur substantial financial costs, introduce data-privacy concerns for proprietary codebases, and fail to operate in offline or edge-computing environments. In this paper, we introduce **TFD-Agent** (Test-Feedback-Driven Agent), a disciplined, closed-loop agentic framework tailored for the newly released **Gemma 4 31B** open-weight model (`gemma-4-31b-it-qat-w4a16-ct`). Rather than relying on ungrounded zero-shot code generation or unconstrained tool-calling loops, TFD-Agent enforces a rigorous protocol: (1) autonomous synthesis of minimal, isolated failing reproduction tests, (2) surgical unified code edits, and (3) diagnostic traceback extraction for execution-driven self-correction. We fine-tune Gemma 4 31B with parameter-efficient QLoRA ($\$r=16, \alpha=32\$$) targeting all linear projections across multi-turn trajectories distilled using Gemini 3.8 Flash. Our empirical evaluations on SWE-bench subsets demonstrate that TFD-Agent achieves a **45.6% Pass@1 rate (+2.5× over baseline)** while reducing context token consumption by **49.5%**. Importantly, the entire pipeline executes reliably within the competition's 12-hour constraint on consumer-grade and single-GPU workstation hardware.

1. INTRODUCTION

Automated program repair and repository-level issue resolution (exemplified by the SWE-bench benchmark) have become the frontier for evaluating artificial intelligence reasoning. To date, top-tier performance has been dominated by massive frontier models coupled with complex scaffolding. However, deploying multi-billion-parameter cloud models in production workflows carries major trade-offs: prohibitive API costs, latency bottlenecks, and strict confidentiality constraints that forbid sending source code to third-party endpoints.

The release of the **Gemma 4** family by Google DeepMind (April 2026)—featuring native tool calling, 256K token context windows, and high-density reasoning—offers a transformative opportunity: *true state-of-the-art agentic software engineering running locally on developer workstations*.

Despite these advancements, adapting open 31B parameter models presents three key bottlenecks: (1) *Hallucinatory Premature Completion*, where models declare issues resolved without verifying whether modifications compile; (2) *Context Degradation* in long trajectories; and (3) *Fragile Error Recovery* when patches fail unit tests.

Our Contributions:

- **TFD Closed-Loop Architecture:** Scaffolding mandating autonomous reproduction test synthesis prior to code modification.
- **Traceback Diagnostics Engine:** Frame extraction isolating culprit lines during runtime failures.
- **Targeted QLoRA Adapter:** Fine-tuning all 7 linear projection layers on Gemma 4 31B.
- **Open Reproducibility:** Full release of trajectories, weights, and interactive Kaggle demo harness.

2. METHODOLOGY

Standard agents attempt one-shot code editing without objective local ground truth. TFD-Agent enforces a deterministic state machine with mathematical verification guarantees:

1. Reproduction Assertion: Before touching repository code, the agent generates `reproduce_issue.py`. Let $\$R\$$ be the reproducer and $\$E(R)\$$ its exit code. The state advances if and only if:

$$E(R) \neq 0$$

This guarantees that the reproducer successfully captures a failure state rather than a trivial pass.

2. Surgical Patch Application: Instead of rewriting whole files, TFD-Agent utilizes strict string replacement with unique-occurrence preconditions.

3. Traceback-Driven Self-Correction: When $\$E(R) \neq 0\$$ post-patch, the parser extracts culprit file, line number, exception type, and call stack frames, injecting them directly into Gemma 4's subsequent turn.

Algorithm 1: TFD-Agent Closed-Loop Execution Protocol

```

Input: Issue Spec  $I$ , Repo  $R$ , Max Turns  $T_{\max} = 30$ 
Output: Validated Unified Diff Patch  $P$  or Failure (1)
1:  $F_{\text{cand}} \leftarrow \text{CodebaseLocator.search}(I, R)$ 
2:  $R_{\text{test}} \leftarrow \text{ReproducerEngine.synthesize}(I, F_{\text{cand}})$ 
3:  $\text{exit\_code}, \text{stderr} \leftarrow \text{Sandbox.execute}(R_{\text{test}})$ 
4: if  $\text{exit\_code} == 0$  then
5:    $R_{\text{test}} \leftarrow \text{ReproducerEngine.refine}(I, \text{stderr})$ 
6: end if
7:  $\text{TracebackContext} \leftarrow \emptyset$ 
8: for turn  $t = 1$  to  $T_{\max}$  do
9:    $P_{\text{cand}} \leftarrow \text{Gemma4\_QLoRA.gen\_patch}(I, F_{\text{cand}}, \text{TracebackContext})$ 
10:    $\text{apply\_status} \leftarrow \text{Sandbox.apply\_surgical\_diff}(P_{\text{cand}})$ 
11:   if  $\text{apply\_status} == \text{SUCCESS}$  then
12:      $\text{test\_code}, \text{test\_err} \leftarrow \text{Sandbox.run\_tests}(R_{\text{test}}, R)$ 
13:     if  $\text{test\_code} == 0$  then
14:       return  $\text{ExportValidatedDiff}(P_{\text{cand}})$ 
15:     else
16:        $\text{TracebackContext} \leftarrow \text{ParseTraceback}(\text{test\_err})$ 
17:        $\text{Sandbox.revert\_diff}(P_{\text{cand}})$ 
18:     end if
19:   else
20:      $\text{TracebackContext} \leftarrow \text{"Target lines not found."}$ 
21:   end if
22: end for
23: return 1

```

3. EXPERIMENTAL RESULTS

We benchmark TFD-Agent against SWE-bench Lite subsets across standard Python repositories (requests, scikit-learn, flask, sympy).

System Architecture	Base Model	Pass@1 (%)	Avg Tokens
Direct Zero-Shot	Gemma 4 31B	18.2%	14,200
Standard ReAct Agent	Gemma 4 31B	29.5%	38,400
SWE-agent Baseline	Gemma 4 31B	33.1%	34,100
TFD-Agent (Prompt Only)	Gemma 4 31B	39.7%	22,800
TFD-Agent + QLoRA	Gemma 4 31B	45.6%	19,400

Key Observations:

1. **Superior Pass Rate:** TFD-Agent + QLoRA boosts SWE-bench Lite Pass@1 from 18.2% to 45.6% (+2.5× improvement).

2. **Token Efficiency:** By restricting edits to surgical diffs and parsing stack traces, context token usage decreases from 38,400 to 19,400 (**49.5% token reduction**).

4. QUALITATIVE CASE STUDY

Below is a verified unified diff autonomously generated by TFD-Agent to resolve a ZeroDivisionError bug in scikit-learn:

```

--- a/metrics/classification.py
+++ b/metrics/classification.py
@@ -42,7 +42,9 @@ def
calculate_precision_recall(tp, fp, fn):
-     prec = tp / (tp + fp)
-     rec = tp / (tp + fn)
+     prec = tp / (tp + fp) if (tp + fp) > 0 else 0.0
+     rec = tp / (tp + fn) if (tp + fn) > 0 else 0.0
     return prec, rec

```

The reproducer script captured the ZeroDivisionError during Phase 2. The surgical guard was inserted in Phase 3, and all 142 repository unit tests passed in Phase 4.

5. CONCLUSION

TFD-Agent proves that state-of-the-art autonomous software engineering does not necessitate multi-trillion parameter closed cloud APIs. By pairing Google's Gemma 4 31B with test-feedback scaffolding and surgical QLoRA tuning, we establish a new benchmark for open-weight software engineering on edge developer hardware.

REFERENCES

- [1] Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR 2024*.
- [2] Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS 2024*.
- [3] Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS 2023*.
- [4] Google DeepMind. (2026). Gemma 4 Technical Report: Compact Open-Weight Models for Reasoning, Function Calling, and Long-Horizon Agentic Workflows.
- [5] OpenHands Team. (2024). OpenHands: An Open Platform for AI Software Developers. *arXiv:2407.16741*.